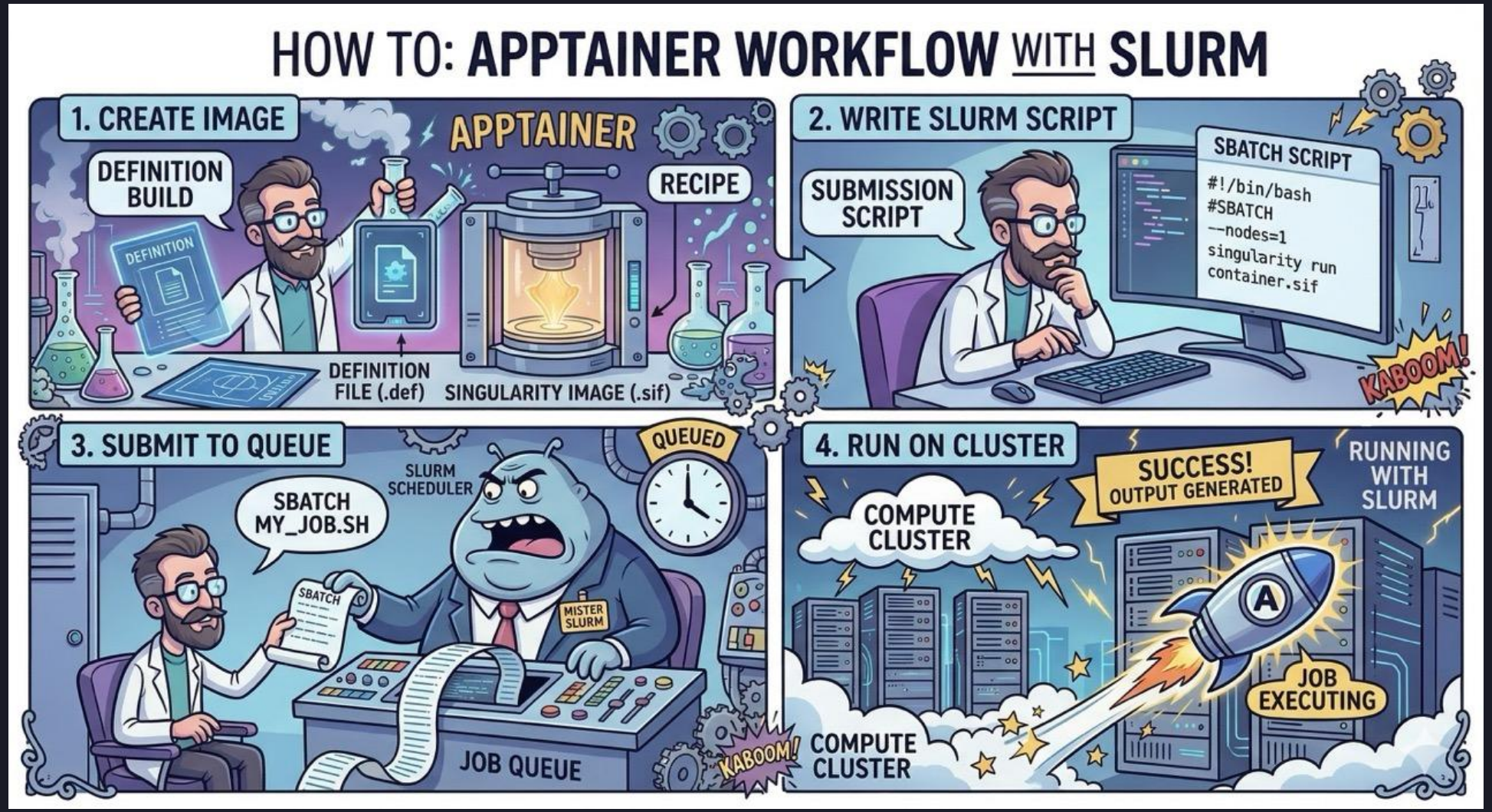
Containers for HPC

Getting Started with Apptainer


HOW TO: APPTAINER WORKFLOW WITH SLURM
1. CREATE IMAGE
DEFINITION BUILD
DEFINITION
APPTAINER
RECIPE
DEFINITION FILE (.def)
SINGULARITY IMAGE (.sif)
2. WRITE SLURM SCRIPT
SUBMISSION SCRIPT
SBATCH SCRIPT
#!/bin/bash
#SBATCH
--nodes=1
singularity run
container.sif
KABOOM!
3. SUBMIT TO QUEUE
SBATCH MY_JOB.SH
SLURM SCHEDULER
QUEUED
SBATCH
MISTER SLURM
JOB QUEUE
KABOOM!
4. RUN ON CLUSTER
COMPUTE CLUSTER
SUCCESS! OUTPUT GENERATED
RUNNING WITH SLURM
JOB EXECUTING
COMPUTE CLUSTER

What Problem Are We Solving?

"It works on my laptop"
The classic research tragedy — your analysis runs fine locally but crashes on the cluster or a collaborator's machine.

Dependency Hell
Conflicting Python, R, or CUDA versions across projects. Installing one package breaks another. Shared clusters can't satisfy everyone.

Irreproducibility
You can't re-run your own analysis 6 months later because the environment has changed. Reviewers can't reproduce your results.

Collaboration Barriers
Sending setup instructions that span 3 pages of README is fragile. One missed step and it falls apart.

✓ Containers solve ALL of these — one file, everything inside it, runs everywhere.

What Is a Container?

📦 The Shipping Container Analogy

- A standardized box that works on any ship, truck, or dock
- Your container works on your laptop, the cluster, and your collaborator's machine
- Bundles your app + ALL its dependencies into ONE portable file
- Isolated from the host — no more "which Python am I using?"
- Lightweight — NOT a full VM, shares the host OS kernel

Container vs. Virtual Machine

Feature	Container	VM
Boot time	< 1 sec	Minutes
Size	MBs	GBs
Overhead	Near zero	Significant
Kernel	Shared	Own copy
HPC ready	✅ Yes	❌ No

> On HPC: Container overhead is < 1-2% for most compute-bound workloads. You pay no meaningful performance tax.

Docker vs. Apptainer: Why Not Just Use Docker?

Feature	Docker	Apptainer
Requires root/sudo	✅ Yes	❌ No
Runs as your user	❌ No	✅ Yes
Security model	Daemon = root risk	Rootless by design
HPC scheduler (SLURM/PBS)	❌ Problematic	✅ Native support
Shared filesystems (Lustre)	⚠️ Issues	✅ Seamless
Import Docker images	N/A	✅ `docker://`
Portable single file	❌ No	✅ `.sif` file

> 🔑 The core problem with Docker on HPC: The Docker daemon runs as root. On a shared cluster with thousands of users and petabytes of data, anyone who can run Docker can trivially escalate to root on the host. HPC sysadmins universally ban it.

Apptainer was built from the ground up in 2015 specifically for shared HPC systems. A user inside a container is always the same user outside.

How Apptainer Works on HPC

1 · Definition File

📄 Plain text recipe
`.def` file in Git
Version controlled

2 · Build Once

🔨 `apptainer build`
Creates `.sif` file
Run as user/fakeroot

3 · One File

📦 Copy, share, store
`rsync` it anywhere
Archive with data

4 · Run Everywhere

🚀 Cluster, cloud, laptop
SLURM native
Same command always

Key Behaviours

- 🔐 Runs under your UID — no privilege escalation ever
- 📂 Auto-mounts `$HOME`, current dir, and `/tmp`
- 💾 Use `--bind` to mount Scratch/Lustre storage
- 🐳 Imports Docker Hub images via `docker://` prefix
- 🖥️ Works natively with SLURM — prefix with `apptainer exec`
- 🔒 Container filesystem is read-only at runtime

Key Apptainer Commands

90% of your day-to-day usage fits in four commands

`apptainer pull` — Download a pre-built image from Docker Hub. Millions available. No build required.

```
apptainer pull python311.sif docker://python:3.11-slim
```

`apptainer shell` — Drop into an interactive shell inside the container. Great for exploring and debugging.

```
apptainer shell myenv.sif
```

`apptainer exec` — Run a single command inside the container. Used in SLURM job scripts.

```
apptainer exec myenv.sif python3 analysis.py
```

`apptainer build` — Build your own image from a definition file. Your reproducibility recipe.

```
apptainer build myenv.sif myenv.def
```

Definition Files: Your Reproducibility Recipe

Why use `.def` files?

- 📄 Version control — check into Git alongside your code
- 🔄 Rebuild anytime — anyone can recreate the exact environment
- 📖 Self-documenting — the `.def` file IS the documentation
- 🤝 Share the recipe — send the `.def`, not the gigabyte `.sif`
- 🔬 Peer review — reviewers can verify and re-run your exact setup

```
# myenv.def

Bootstrap: docker        # start from Docker Hub
From: ubuntu:22.04       # base image

%post                    # runs during BUILD (as root)
    apt-get install -y python3
    pip install numpy scipy

%environment             # set at RUNTIME
    export PATH=/opt/app:$PATH
    export LC_ALL=C

%runscript               # default command
    python3 analysis.py
```

> 💡 Key insight: Bootstrap + From tap the entire Docker Hub ecosystem. Start from Python, R, Ubuntu, CUDA, Bioconductor — whatever fits your project.

Real Research Use Cases

🧬 Bioinformatics

- Pin exact GATK, BWA, Samtools versions
- Share complete pipeline with your paper
- Run legacy tools on modern clusters
- Increasingly required by journals (Nature Methods)

🧠 ML / Deep Learning

- Freeze CUDA + PyTorch + driver versions
- Guarantee GPU code reproduces exactly
- Move between GPU cluster generations seamlessly

🐍 Python / R Science

- No more conda environment conflicts
- Isolate per-project dependencies
- Collaborate across lab members safely

🗄️ Legacy Code & Archive

- Run 2015 workflows on 2025 clusters
- Archive containers alongside published data
- Meet journal reproducibility requirements

> 💡 Pattern: Build once → archive the .sif with your data → anyone re-runs your analysis years later with zero setup friction.

Typical HPC Workflow

```
┌──────────────┐   ┌──────────────┐   ┌──────────────┐   ┌──────────────┐
│  1. BUILD    │──▶│  2. STORE    │──▶│  3. TEST     │──▶│  4. RUN      │
│              │   │              │   │              │   │              │
│ apptainer build│ │ $HOME/containers│ │ apptainer shell│ │ sbatch job.sh│
│ myenv.sif    │   │ /myenv.sif   │   │ myenv.sif    │   │ (apptainer exec│
│ myenv.def    │   │              │   │              │   │  inside)     │
└──────────────┘   └──────────────┘   └──────────────┘   └──────────────┘
```

Typical SLURM job script

```bash
#!/bin/bash
#SBATCH --job-name=myanalysis
#SBATCH --cpus-per-task=8
#SBATCH --mem=32G
#SBATCH --time=04:00:00
#SBATCH --output=logs/%j.out

apptainer exec \
    --bind $SCRATCH:/scratch \
    $HOME/containers/myenv.sif \
    python3 /scratch/my_analysis.py
```

> 💡 The scheduler sees a normal job. apptainer exec is completely transparent to SLURM — no special configuration needed.

Tips & Gotchas

🔧 Building — needs root or a workaround

```
# Option 1: fakeroot (most clusters)
apptainer build --fakeroot myenv.sif myenv.def

# Option 2: build remotely (free, no root)
# Upload .def to cloud.sylabs.io
# Download finished .sif
```

🔒 Container filesystem is READ-ONLY

```
# WRONG — can't write inside the container
apptainer exec myenv.sif touch /opt/newfile

# RIGHT — write to bound host paths
apptainer exec --bind $SCRATCH:/scratch \
    myenv.sif python3 script.py
#   (script writes to /scratch/results)
```

🖥️ GPU Jobs — the `--nv` flag

```
# Pass through NVIDIA GPU from host
apptainer exec --nv pytorch.sif python3 train.py

# In SLURM: request GPU first
#SBATCH --gres=gpu:1
apptainer exec --nv pytorch.sif python3 train.py
```

💾 Large Image Cache

```
# Default cache can fill your home quota
# Redirect to scratch:
export APPTAINER_CACHEDIR=$SCRATCH/.appcache

# Inspect any image
apptainer inspect --deffile image.sif
apptainer inspect --labels  image.sif
```

HANDS-ON DEMOS

Demo 1 — Pull & Explore a Pre-built Image

Step 1 — Pull Python 3.11 from Docker Hub

```
# Pull Python from Docker Hub (~200MB, takes 1-2 min)
apptainer pull python311.sif docker://python:3.11-slim
echo "Pull complete. File size:"
ls -lh python311.sif 2>/dev/null || echo "(run on your cluster)"
```

[finished]

```
/var/folders/5c/zr4zrjp56sb13hl6x0khl7t00000gq/T/.presentermPJovAS/script.sh: line 2: apptainer: command not
found
Pull complete. File size:
(run on your cluster)
```

Step 2 — Compare host vs container Python

```
# Your host Python
python3 --version

# Container Python (different version, isolated)
apptainer exec python311.sif python3 --version
```

[finished with error]

```
Python 3.11.9
/var/folders/5c/zr4zrjp56sb13hl6x0khl7t00000gq/T/.presentermPZ3zzu/script.sh: line 5: apptainer: command not
found
```

Demo 1 — Pull & Explore a Pre-built Image

Step 3 — Drop into an interactive shell

```
apptainer shell python311.sif
# Apptainer> whoami          ← same username as host!
# Apptainer> echo $HOME      ← your home dir is mounted
# Apptainer> pip list        ← see installed packages
# Apptainer> exit
```

> 🗝🗝Notice: Same username inside and outside. $HOME already mounted. No sudo anywhere.

Demo 2 — Build a Simple Science Image

⏱ 10 min · "Your first definition file"

scipy-demo.def

```
Bootstrap: docker
From: python:3.11-slim

%post
    pip install --no-cache-dir \
        numpy scipy matplotlib

%runscript
    python3 -c "
import numpy as np
from scipy import stats
data = np.random.normal(5, 2, 1000)
print(f'Mean:  {data.mean():.3f}')
print(f'Stdev: {data.std():.3f}')
t, p = stats.ttest_1samp(data, 5)
print(f't={t:.3f}  p={p:.4f}')
"
```

Build & run

```
# Write the def file
nano scipy-demo.def

# Build the container
apptainer build \
    scipy-demo.sif \
    scipy-demo.def

# Check file size
ls -lh scipy-demo.sif

# Run the %runscript
apptainer run scipy-demo.sif

# Confirm container Python version
apptainer exec scipy-demo.sif \
    python3 --version
```

🎯 Reproducibility magic: Send the 300MB .sif to anyone → identical output, same numpy/scipy versions, guaranteed.

Demo 3 — Bioinformatics Pipeline Container

⏱ 10 min · "Real-world research example — FastQC + MultiQC"

bioinfo.def

```
Bootstrap: docker
From: ubuntu:22.04

%post
    apt-get update -y
    apt-get install -y \
        python3 python3-pip \
        wget curl fastqc
    pip3 install multiqc

%environment
    export LC_ALL=C   # required!
    export LANG=C

%runscript
    echo "=== Bioinformatics Tools ==="
    fastqc --version
    multiqc --version
```

Build & run on data

```
# Build (~3-4 min — more packages)
apptainer build \
    bioinfo.sif bioinfo.def

# Verify tools installed
apptainer run bioinfo.sif

# Run FastQC on your data
# (--bind makes ./data visible as /data)
apptainer exec \
    --bind ./data:/data \
    bioinfo.sif \
    fastqc /data/sample.fastq.gz \
    -o /data/results/

# Aggregate with MultiQC
apptainer exec \
    --bind ./data:/data \
    bioinfo.sif \
    multiqc /data/results/
```

🔑 The bind mount pattern: Container is read-only. All I/O happens through --bind paths. Your data never enters the container image.

Demo 4 — SLURM Job Script Integration

⏱ 7 min · "Submitting container jobs to the scheduler"

run_analysis.sh

```
#!/bin/bash
#SBATCH --job-name=apptainer-demo
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=4          # 4 CPU cores
#SBATCH --mem=8G                   # 8 GB RAM
#SBATCH --time=01:00:00            # 1 hour walltime
#SBATCH --output=logs/%j.out      # log per job ID

# Run analysis inside container
apptainer exec \
    --bind $SCRATCH:/scratch \          # mount Lustre scratch
    $HOME/containers/scipy-demo.sif \   # your .sif file
    python3 /scratch/analysis/my_analysis.py
```

Submit

```
mkdir -p logs
sbatch run_analysis.sh
```

Monitor

```
squeue -u $USER
```

Cancel

```
scancel <jobid>
```

💡 SLURM sees a normal job. No special configuration. Just prefix your command with apptainer exec — the rest is identical to any other batch script.

Demo 5 — GPU Container: PyTorch + CUDA

⏱ 5 min · "NVIDIA GPU passthrough — the --nv flag"

pytorch-gpu.def

```
Bootstrap: docker
From: pytorch/pytorch:2.2.0-cuda12.1-cudnn8-runtime

%post
    pip install --no-cache-dir \
        torchvision torchaudio

%runscript
    python3 -c "
import torch
print('PyTorch:', torch.__version__)
print('CUDA available:', torch.cuda.is_available())
if torch.cuda.is_available():
    gpu = torch.cuda.get_device_name(0)
    print('GPU:', gpu)
"
```

Run with GPU passthrough

```
# Build (5+ min — large base image)
apptainer build \
    pytorch.sif pytorch-gpu.def

# Without --nv: no GPU access
apptainer run pytorch.sif
# → CUDA available: False

# With --nv: GPU passes through!
apptainer run --nv pytorch.sif
# → CUDA available: True
# → GPU: NVIDIA A100 80GB

# In a SLURM script:
# #SBATCH --gres=gpu:1
apptainer exec --nv \
    pytorch.sif python3 train.py
```

🎮 --nv magic: The flag binds NVIDIA drivers from the host into the container. No CUDA installation needed inside the image — it inherits the host's GPU driver automatically.

Key Takeaways & Resources

Key Takeaways

- 📦 One `.sif` file = your entire software environment, frozen in time Not a requirements.txt — the `.sif` IS the environment, ready to run
- 👤 Runs as YOU — same UID inside and outside. No root, no risk. This is why sysadmins allow it — it can't escalate privileges
- 🌊 The entire Docker Hub ecosystem works via `docker://` prefix Millions of images: Python, R, CUDA, Bioconductor, GATK, and more
- 📄 `.def` files in Git = reproducible, peer-reviewable science Your environment is code — version control it like code
- 🖥️ Transparent to SLURM — just prefix with `apptainer exec` No special scheduler config, no sysadmin intervention needed

Resources

Resource	URL	Notes
📖 Apptainer docs	apptainer.org/docs	Official — installation, CLI reference
☁️ Sylabs Cloud	cloud.sylabs.io	Free remote builds — no root needed
🐳 Docker Hub	hub.docker.com	Use with docker:// prefix
🔑 Def file examples	github.com/apptainer/apptainer	Community definition files

Quick reference card

```
apptainer pull  myenv.sif  docker://image:tag   # download image
apptainer shell myenv.sif                        # interactive shell
apptainer exec  myenv.sif  <command>             # run one command
apptainer build myenv.sif  myenv.def             # build from recipe
apptainer exec  --bind /scratch:/scratch  ...    # mount filesystem
apptainer exec  --nv  ...                        # enable NVIDIA GPU
apptainer inspect --deffile myenv.sif            # inspect image
```

Questions? Reach out with your specific workflow — every container problem has a solution.